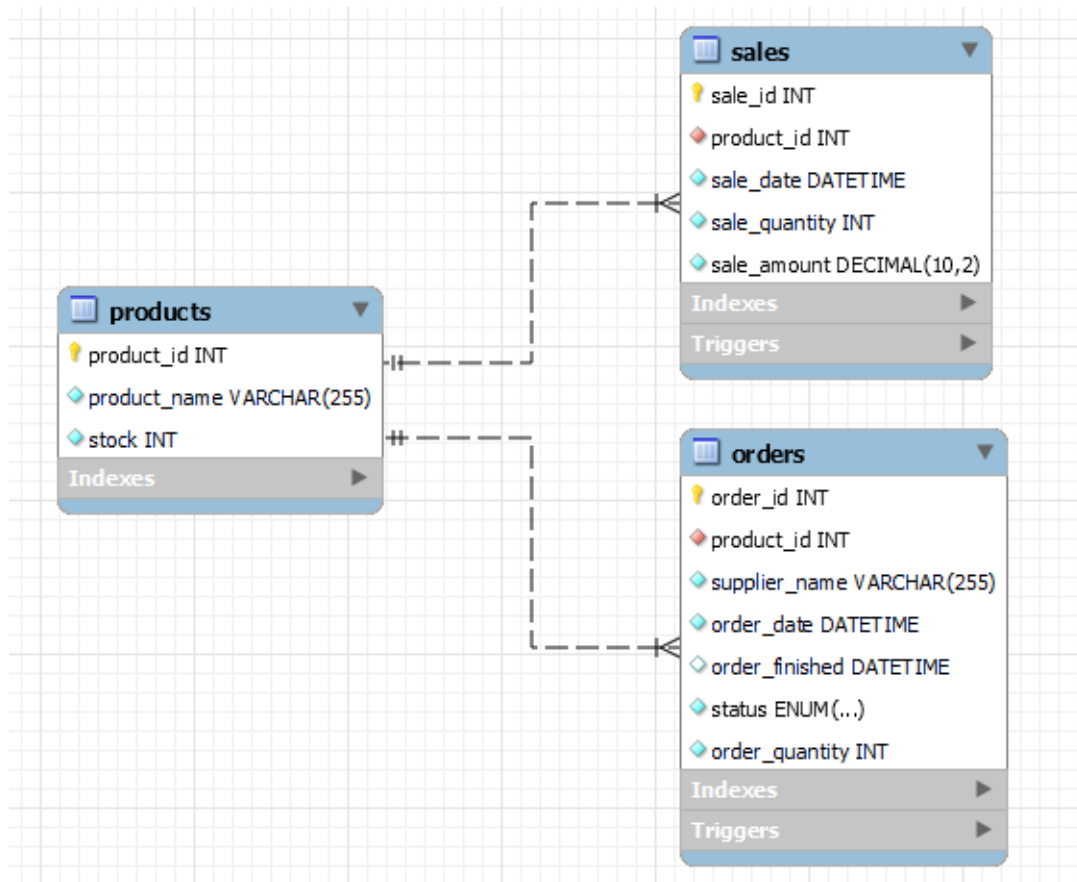


Se tiene un sistema de ventas e inventario muy sencillo, con sólo 3 tablas: Products, Orders y Sales.

- La tabla **Products** tiene el registro de los productos y su stock disponible.
- La tabla **Orders** tiene el registro de los pedidos de productos que se hacen a los proveedores.
- La tabla **Sales** tiene el registro de las ventas.

El modelo de datos es el siguiente:



La tabla Products no tiene registrado el precio de venta, ya que la empresa es muy flexible en ello, y aunque tiene una tabla de precios que maneja de manera manual, puede negociar caso a caso con los clientes.

INFO209 – Taller de Bases de Datos – SQL
16-abril-2026

Cree tablas y cargue datos en el **schema INFO209**. El script de creación de las tablas y carga de datos es el siguiente:

```
CREATE TABLE Products (  
    product_id INT UNSIGNED AUTO_INCREMENT,  
    product_name VARCHAR(255) not null,  
    stock INT UNSIGNED NOT NULL DEFAULT 0,  
    PRIMARY KEY (product_id)  
);  
  
CREATE TABLE Orders (  
    order_id INT not null AUTO_INCREMENT,  
    product_id INT UNSIGNED not null,  
    supplier_name VARCHAR(255) not null,  
    order_date DATETIME not null,  
    order_finished DATETIME,  
    status ENUM('SENT', 'RECEIVED', 'CANCELLED') NOT NULL DEFAULT 'SENT',  
    order_quantity INT UNSIGNED not null,  
    PRIMARY KEY (order_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);  
  
CREATE TABLE Sales (  
    sale_id INT UNSIGNED AUTO_INCREMENT,  
    product_id INT UNSIGNED NOT NULL,  
    sale_date DATETIME NOT NULL,  
    sale_quantity INT UNSIGNED NOT NULL,  
    sale_amount DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (sale_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);  
  
Insert into Products (product_name, stock) values ('Pilas AA - Alcalina', 50);  
Insert into Products (product_name, stock) values ('Pilas AAA - Alcalina', 10);  
Insert into Products (product_name, stock) values ('Pilas CR2025', 15);  
Insert into Products (product_name, stock) values ('Pilas CR2032', 5);  
  
Insert into Orders  
(product_id, supplier_name, order_date, order_quantity)  
values (1, 'Duracell Chile', now(), 200);  
Insert into Orders  
(product_id, supplier_name, order_date, order_quantity)  
values (2, 'Duracell Chile', now(), 100);  
Insert into Orders  
(product_id, supplier_name, order_date, order_quantity)  
values (3, 'Duracell Chile', now(), 20);  
  
Insert into Sales  
(sale_id, product_id, sale_date, sale_quantity, sale_amount)  
Values (1, 1, now(), 5, 1500);  
  
Insert into Sales  
(sale_id, product_id, sale_date, sale_quantity, sale_amount)  
Values (2, 3, now(), 2, 2000);
```

En la sesión pasada, vimos que es posible **manejar excepciones** en procedimientos almacenados, a nivel general (**SQLException**). Este ejemplo muestra que podemos manejar excepciones específicas, utilizando los códigos de error.

Por ejemplo, si ejecutamos la instrucción: insert into transferencias values (1,5,100, now());

Devuelve el siguiente error, en caso de no existir una cuenta (**violación de llave foránea**):

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`info209`.`transferencias`, CONSTRAINT `fk\_destino` FOREIGN KEY (`id\_cuenta\_destino`) REFERENCES `cuentas` (`id\_cuenta`))

Podemos, entonces, utilizar el código de error respectivo dentro del procedimiento almacenado:

```
DELIMITER $$
DROP PROCEDURE IF EXISTS p_transferir$$
CREATE PROCEDURE p_transferir(
    IN cta_origen int, IN cta_destino int, IN monto int, OUT mensaje varchar (100)
)
BEGIN
    declare exit handler for 1452
    begin
        set mensaje = 'transferencia involucra cuenta inexistente';
        rollback;
    end;
    declare exit handler for SQLException
    begin
        set mensaje = 'transacción no realizada';
        rollback;
    end;
    start transaction;
    insert into transferencias values (cta_origen, cta_destino, monto, now());
    update cuentas set saldo = saldo - monto where id_cuenta = cta_origen;
    update cuentas set saldo = saldo + monto where id_cuenta = cta_destino;
    commit;
end
$$
DELIMITER ;
```

¿Cómo ejecutamos este procedimiento en la línea de comando, para ver el mensaje de salida?

```
set @mensaje = '0';
call info209.p_transferir(1, 5, 100, @mensaje);
select @mensaje;
```

1. Implementar procedimientos almacenados.

Utilizando las tablas proporcionadas (Orders, Products, Sales), se le pide implementar procedimientos almacenados para realizar las siguientes acciones:

- **Registrar una venta.** Los parámetros **de entrada** son los siguientes: product_id, sale_quantity, sale_amount. Esto implica:
 - a. Una inserción en la tabla Sales.
 - b. Una rebaja de inventario en la tabla Products.
- **Registrar la llegada de productos** ya solicitados a un proveedor. El parámetro **de entrada** es el order_id. Esto implica:
 - a. Actualizar la tabla Orders, colocando now() en order_finished, y actualizar el status con 'RECEIVED', sólo si la orden está en estado 'SENT'. No pueden recibirse productos de órdenes ya recibidas, ni tampoco de órdenes canceladas.
 - b. Incrementar el stock en la tabla Products, con la cantidad de la orden.

Recuerde que **ambos procedimientos requieren un adecuado manejo de transacciones**. Esto implica considerar las propiedades ACID, y también que pueden ocurrir errores, que implicarán deshacer la transacción. Ejemplos de errores (la lista es sólo ilustrativa):

- Registrar una venta para un producto que no existe, debiera producir violación de llave foránea.
- La rebaja de stock al registrar una venta implica stock negativo (no debe permitirse).
- No debe permitirse la llegada de productos de una orden inexistente.

En concreto, se le pide entonces realizar lo siguiente:

- a) Identificar **al menos dos códigos de excepción** a considerar en el procedimiento almacenado de registro de una venta.
- b) Implementar el procedimiento almacenado de registro de una venta.
- c) Mostrar que funciona el registro de una venta correcta.
- d) Mostrar que, si el procedimiento es ejecutado de modo que pudiese generar stock negativo, devuelve un mensaje de error y no registra la venta.
- e) Identificar **al menos dos códigos de excepción** a considerar en el procedimiento almacenado de registro de llegada de productos (pueden ser los mismos anteriores o diferentes).
- f) Implementar el procedimiento almacenado de registro de llegada de productos.
- g) Mostrar que funciona el registro de llegada de productos de una order_id correcta.
- h) Mostrar que, si el procedimiento es ejecutado con un order_id inexistente, devuelve un mensaje de error y no registra la llegada de productos.

2. Funciones.

Como ya han podido ver, el lenguaje SQL permite utilizar funciones en operaciones de manipulación de datos. Por ejemplo, han usado COUNT(*) y SUM(atributo) desde un SELECT, y también NOW() en INSERT o UPDATE.

La posibilidad anterior no se restringe a funciones predefinidas, sino que nosotros podemos definir nuestras propias funciones.

¿Qué utilidad tiene definir funciones? En general, sirven para reducir la complejidad de las sentencias SQL y para facilitar la reutilización de funcionalidades. Imagine que debe reunir información desde muchas tablas en una consulta. Para ello, podría decidir:

- Hacer joins entre dichas tablas.
- Tomar una tabla central y crear funciones para ir a buscar la información de las otras tablas.
- Una combinación de lo anterior.

Para crear funciones en un usuario diferente a Root, debe dar a su usuario el privilegio **SUPER** (como **root**, vaya a Administrative Roles / Global Privileges).

Veamos un ejemplo:

```
DELIMITER $$
drop function if exists f_PrecioPromedio$$
CREATE FUNCTION f_PrecioPromedio(producto_id INT) RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE precio_promedio DECIMAL(10,2);
    SELECT AVG(sale_amount / sale_quantity) INTO precio_promedio
    FROM Sales
    WHERE product_id = producto_id;

    RETURN precio_promedio;
END$$
DELIMITER ;
```

Se pide crear una **vista** que obtenga el listado de productos, con su nombre y el precio promedio cobrado en las ventas.

- Compile la función proporcionada, f_PrecioPromedio.
- (2a) Pida a Chat GPT (u otra IA), que le explique qué hace esa función, línea a línea.
 - o ¿Por qué se incluye la línea READS SQL DATA?
 - o ¿Por qué se usa el delimitador \$\$?
- (2b) Utilice la función para **crear una vista** que entregue la información solicitada, sin realizar joins. Haga SELECT sólo a la tabla PRODUCTS.
- (2c) Haga una **consulta** a los datos de la vista, con alguna condición en el WHERE.

(2d) Cree una **función** que reciba como parámetro el identificador del producto y el **status** de la orden, y retorne la cantidad de productos que tienen ese **status** en la tabla ORDERS.

(2e) **Utilice la función** creada en el paso anterior **para modificar la vista**, agregando una columna que entregue la cantidad de productos que tienen STATUS = 'SENT'; es decir, que fueron solicitados y aún no han llegado.

- Considere usar la función IFNULL(atributo, valor), que entrega un valor definido si el atributo tiene valor nulo. En este caso, si no hay productos en estado 'SENT', debe entregar 0.

3. Triggers.

En clases vimos que los triggers nos permiten implementar logs y validaciones sobre operaciones de inserción, actualización y borrado de registros en las tablas.

Para realizar una validación, se pueden implementar triggers que generen un error cuando se realiza una operación no permitida.

Por ejemplo, en la tabla de transferencias que utilizamos en las semanas previas, podemos evitar que se inserten transferencias donde la cuenta de origen y la de destino sean la misma, mediante este trigger:

```
DELIMITER $$

CREATE TRIGGER trg_transferencias_no_misma_cuenta
BEFORE INSERT ON transferencias
FOR EACH ROW
BEGIN
IF NEW.id_cuenta_origen = NEW.id_cuenta_destino THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'La cta de origen no puede igual a la cta de destino';
END IF;
END$$

DELIMITER ;
```

- a) **Compile el trigger y pruebe que funciona**, mediante una instrucción SQL que lo ponga a prueba.
- b) **Implemente un trigger** sobre la tabla de Orders, que impida la modificación del estado de la orden, en caso de que esta se encuentre ya recibida o haya sido cancelada. **Pruebe que funciona**, mediante una instrucción update.